

ПРАКТИЧЕСКОЕ ЗАНЯТИЕ 4.1

Тема: *Бинарная классификация текстов с использованием нейронной сети Keras (набор данных IMDb)*

Длительность: 2 академических часа

Инструменты: Python, Jupyter Notebook, TensorFlow/Keras

1. Цели работы

После выполнения данной работы студент должен:

1. Понять, что такое задача бинарной классификации.
2. Научиться загружать и подготавливать текстовые данные.
3. Освоить преобразование данных в числовое представление.
4. Построить первую полносвязную нейронную сеть Keras.
5. Выполнить обучение модели и построить графики качества.
6. Научиться интерпретировать результаты модели.
7. Провести собственное исследование и сформулировать выводы.

2. Краткие теоретические сведения

2.1. Бинарная классификация

Это задача, в которой необходимо разделить объекты на **два класса**.
В нашем примере:

- вход — текстовая рецензия на фильм,
- выход — *положительная* или *отрицательная* рецензия.

Модель должна научиться по тексту определять эмоциональную окраску.

2.2. Набор данных IMDb

База содержит **50 000** текстовых отзывов, разделённых поровну на положительные и отрицательные.

Keras предоставляет готовую версию IMDb, в которой:

- каждый отзыв превращён в последовательность индексов слов,
- словарь ограничен несколькими тысячами наиболее частых слов,
- данные уже разделены на обучающую и тестовую выборки.

2.3. Подготовка данных

Нейронные сети работают только с числовыми массивами фиксированной длины, поэтому тексты необходимо:

1. **Токенизировать** (превратить в последовательность индексов).
2. **Векторизовать** (создать числовой вектор фиксированной длины).

В этой работе используем подход **многомерного one-hot кодирования**, где каждый отзыв превращается в вектор из 0/1 длины 10000.

2.4. Архитектура модели

Мы построим простую модель:

```
Dense(16, activation='relu')
Dense(16, activation='relu')
Dense(1, activation='sigmoid')
```

- relu помогает выучивать нелинейные зависимости,
- sigmoid даёт вероятность класса (от 0 до 1),
- функция потерь — **binary_crossentropy**.

2.5. Переобучение

Простой пример переобучения:

- ошибка на обучении ↓↓
- ошибка на валидации начинает расти ↑↑

Мы построим графики и проанализируем этот эффект.

3. Порядок выполнения работы

Каждый шаг будет продублирован в Jupyter Notebook.

Шаг 1. Подготовка окружения

Убедиться, что установлены:

```
pip install tensorflow matplotlib numpy
```

Открыть ноутбук.

Шаг 2. Загрузка набора данных IMDB

Студент выполняет:

```
from tensorflow.keras.datasets import imdb
```

```
(train_data, train_labels), (test_data, test_labels) =  
imdb.load_data(num_words=10000)
```

Объяснение:

num_words=10000 означает, что мы оставляем только 10 000 наиболее частых слов.

Каждый отзыв — список чисел, например:

```
[1, 14, 20, 8, 176, 2, 34, ...]
```

Шаг 3. Декодирование отзыва (демонстрация)

Студент получает текстовую форму:

```
word_index = imdb.get_word_index()  
reverse_word_index = {value: key for (key, value) in word_index.items()}  
decoded_review = " ".join([reverse_word_index.get(i - 3, '?') for i in  
train_data[0]])  
print(decoded_review)
```

В методичке объясняется:

- что означает сдвиг индексов,
- почему текст выглядит «шумным».

Шаг 4. Векторизация данных

Студент создаёт функцию:

```
import numpy as np
```

```
def vectorize_sequences(sequences, dimension=10000):  
    results = np.zeros((len(sequences), dimension))  
    for i, seq in enumerate(sequences):  
        results[i, seq] = 1.0  
    return results
```

```
x_train = vectorize_sequences(train_data)  
x_test = vectorize_sequences(test_data)
```

Объяснение:

- данные должны иметь одинаковую размерность,
- каждый отзыв → вектор признаков длиной 10000.

Метки тоже переводятся в float32.

Шаг 5. Формирование валидационной выборки

```
x_val = x_train[:10000]
partial_x_train = x_train[10000:]

y_val = train_labels[:10000]
partial_y_train = train_labels[10000:]
```

Объясняется роль валидации.

Шаг 6. Построение модели

```
from tensorflow import keras
from tensorflow.keras import layers

model = keras.Sequential([
    layers.Dense(16, activation="relu"),
    layers.Dense(16, activation="relu"),
    layers.Dense(1, activation="sigmoid")
])

model.compile(
    optimizer="rmsprop",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)
```

Объяснение:

- почему именно такие слои,
- зачем нужна функция активации,
- смысл функции потерь.

Шаг 7. Обучение модели

```
history = model.fit(
    partial_x_train,
    partial_y_train,
    epochs=20,
    batch_size=512,
    validation_data=(x_val, y_val)
)
```

Студент наблюдает:

- loss/accurasy на обучении,

- loss/accuracy на валидации.

Шаг 8. Визуализация результатов

Студент строит графики:

- потери на обучении и валидации,
- точности на обучении и валидации.

И делает выводы о переобучении.

Шаг 9. Финальная модель

Тренируем с оптимальным числом эпох (обычно 4–5).

Шаг 10. Оценка модели

```
results = model.evaluate(x_test, test_labels)
print(results)
```

Шаг 11. Прогнозирование новых данных

```
model.predict(x_test[:3])
```

Студент интерпретирует вероятности:

- 0.5 → положительный отзыв,
- <0.5 → отрицательный.

4. Индивидуальные исследовательские задания

Каждый студент получает **2 задания** (по номеру по списку и +1).

Примеры:

1. Изменить размер скрытых слоёв (32, 64, 128) и сравнить графики.
2. Использовать tanh вместо relu и оценить влияние.
3. Добавить слой Dropout и сравнить результаты.
4. Изменить размер словаря (num_words).
5. Изменить функцию оптимизатора (SGD, Adam).
6. Использовать L2-регуляризацию.
7. Изменить размер батча.
8. Ограничить длину текста (обрезка).
9. Усреднить прогнозы нескольких моделей.
10. Сделать baseline-модель (логистическая регрессия).
11. Попробовать обучение 3–4 эпохами вместо 20.
12. Изменить разбиение на валидацию.
13. Построить confusion matrix.

14. Исследовать влияние нормализации признаков.
15. Добавить раннюю остановку (EarlyStopping).

Каждый студент должен представить мини-отчёт.

5. Контрольные вопросы (каждому студенту — любые 2)

1. Почему необходимо векторизовать текстовые данные?
2. В чём разница между функцией активации relu и sigmoid?
3. Для чего используется валидационная выборка?
4. Как распознать переобучение?
5. Что означает значение выхода 0.85 модели?
6. Почему при бинарной классификации используется `binary_crossentropy`?
7. Что означает параметр `num_words` при загрузке IMDB?
8. Что делает функция потерь?

6. Что должен сохранить студент

По итогам работы студент сохраняет:

- Jupyter Notebook с выполненными шагами;
- графики обучения;
- ответы на контрольные вопросы;
- мини-отчёт по исследовательским заданиям (1–2 страницы).

```
{
  "cells": [
    {
      "cell_type": "markdown",
      "metadata": {},
      "source": [
        "# Практическое занятие 4.1\n",
        "## Бинарная классификация текстов с использованием нейронной сети Keras (набор данных IMDb)\n",
        "\n",
        "**Цели работы:**\n",
        "\n",
        "- Познакомиться с задачей бинарной классификации на примере анализа тональности отзывов о фильмах.\n",
        "- Научиться загружать и подготавливать текстовые данные из встроенного набора данных IMDb в Keras.\n",
        "- Освоить преобразование текстов в числовое представление (vectorization).\n",
        "- Построить и обучить простую полносвязную нейронную сеть для бинарной классификации.\n",
        "- Научиться визуализировать процесс обучения и анализировать переобучение.\n",
        "- Выполнить небольшое исследование (изменение архитектуры/гиперпараметров) и сформулировать выводы."
      ]
    },
    {
      "cell_type": "markdown",
      "metadata": {},
      "source": [
        "## 1. Подготовка окружения\n",
        "\n",
        "В этой части мы проверим, что у нас установлены необходимые библиотеки.\n",
        "\n",
        "- `tensorflow` / `keras` — для построения и обучения модели;\n",
        "- `numpy` — для работы с массивами;\n",
        "- `matplotlib` — для визуализации результатов.\n",
        "\n",
        "Если каких-то библиотек не хватает, их необходимо установить перед началом работы (через `pip install ...`)."
      ]
    },
    {
      "cell_type": "code",
      "execution_count": null,
      "metadata": {},
      "outputs": [],
      "source": [
        "# 1. Импорт необходимых библиотек\n",

```

```

        "\n",
        "import numpy as np\n",
        "import matplotlib.pyplot as plt\n",
        "\n",
        "from tensorflow.keras.datasets import imdb\n",
        "from tensorflow import keras\n",
        "from tensorflow.keras import layers\n",
        "\n",
        "print(\"Готово: библиотеки успешно импортированы.\")"
    ]
},
{
    "cell_type": "markdown",
    "metadata": {},
    "source": [
        "## 2. Описание задачи и набора данных IMDb\n",
        "\n",
        "Мы будем работать с задачей **бинарной классификации**:\n",
        "- вход: текстовая рецензия на фильм (отзыв),\n",
        "- выход: метка класса (1 – положительный отзыв, 0 – отрицательный отзыв).\n",
        "\n",
        "Набор данных **IMDb**, встроенный в Keras, содержит:\n",
        "- 50 000 отзывов,\n",
        "- данные уже предобработаны: каждый отзыв представлен как последовательность индексов слов,\n",
        "- есть разделение на обучающую и тестовую выборку.\n",
        "\n",
        "Мы ограничим словарь первыми `num_words = 10000` наиболее часто встречающимися словами."
    ]
},
{
    "cell_type": "code",
    "execution_count": null,
    "metadata": {},
    "outputs": [],
    "source": [
        "# 2. Загрузка набора данных IMDb\n",
        "\n",
        "num_words = 10000 # размер словаря: используем 10 000 наиболее частых слов\n",
        "\n",
        "(train_data, train_labels), (test_data, test_labels) =\n",
        "imdb.load_data(num_words=num_words)\n",
        "\n",
        "print(\"Количество обучающих примеров:\", len(train_data))\n",
        "print(\"Количество тестовых примеров:\", len(test_data))\n",
        "\n",
        "print(\"Пример закодированного отзыва (первые 20 индексов):\")
    ]
}

```

```

        "print(train_data[0][:20])\n",
        "\n",
        "print(\"Метка первого отзыва:\", train_labels[0])"
    ]
},
{
    "cell_type": "markdown",
    "metadata": {},
    "source": [
        "## 3. Анализ формы данных\n",
        "\n",
        "Обратите внимание:\n",
        "\n",
        "- `train_data` — это список отзывов; каждый отзыв — список целых чисел (индексов слов).\n",
        "- Длина отзывов различна (некоторые длиннее, некоторые короче).\n",
        "- `train_labels` — это список меток (0 или 1).\n",
        "\n",
        "***Задача:** привести данные к виду, который можно будет подать на вход нейронной сети:\n",
        "все объекты должны иметь одинаковую размерность."
    ]
},
{
    "cell_type": "code",
    "execution_count": null,
    "metadata": {},
    "outputs": [],
    "source": [
        "# 3. Декодирование отзыва в текст (для наглядности)\n",
        "\n",
        "# загружаем словарь: слово -> индекс\n",
        "word_index = imdb.get_word_index()\n",
        "\n",
        "# разворачиваем словарь: индекс -> слово\n",
        "reverse_word_index = {value: key for (key, value) in\nword_index.items()}\n",
        "\n",
        "# индексы 0, 1, 2 зарезервированы (padding, start и т.п.), поэтому\nвычитаем 3\n",
        "decoded_review = \" \".join([reverse_word_index.get(i - 3, "?")\nfor i in train_data[0]])\n",
        "\n",
        "print(\"Пример декодированного отзыва:\\n\")\n",
        "print(decoded_review)"
    ]
},
{
    "cell_type": "markdown",
    "metadata": {},

```

```

        "source": [
            "## 4. Векторизация последовательностей\n",
            "\n",
            "Нейронная сеть ожидает на вход **числовые векторы фиксированной\n",
            "длины**.\n",
            "Мы используем простую схему:\n",
            "\n",
            "- каждая рецензия превращается в вектор длины `num_words` (10\n",
            "000).\n",
            "- в  $i$ -й позиции стоит 1, если слово с индексом  $i$  встречается в\n",
            "отзыве, и 0 – если нет.\n",
            "\n",
            "Это вариант **многомерного one-hot представления** (bag-of-words)."]
    },
    {
        "cell_type": "code",
        "execution_count": null,
        "metadata": {},
        "outputs": [],
        "source": [
            "# 4. Функция векторизации последовательностей\n",
            "\n",
            "def vectorize_sequences(sequences, dimension=10000):\n",
            "    \"\"\"\n",
            "        Преобразует список последовательностей (списки индексов\n",
            "слов)\n",
            "        в двумерный numpy-массив размера (количество_примеров,\n",
            "dimension),\n",
            "        где каждый отзыв представлен в виде вектора 0/1.\n",
            "    \"\"\"\n",
            "    results = np.zeros((len(sequences), dimension),\n",
            "dtype=\"float32\")\n",
            "    for i, seq in enumerate(sequences):\n",
            "        results[i, seq] = 1.0 # выставляем 1.0 в позициях индексов\n",
            "слов\n",
            "    return results\n",
            "\n",
            "x_train = vectorize_sequences(train_data, dimension=num_words)\n",
            "x_test = vectorize_sequences(test_data, dimension=num_words)\n",
            "\n",
            "# Преобразуем метки в массив float32\n",
            "y_train = np.asarray(train_labels).astype(\"float32\")\n",
            "y_test = np.asarray(test_labels).astype(\"float32\")\n",
            "\n",
            "print(\"Форма x_train:\", x_train.shape)\n",
            "print(\"Форма x_test:\", x_test.shape)\n",
            "print(\"Форма y_train:\", y_train.shape)\n",
            "print(\"Форма y_test:\", y_test.shape)"]
    ]

```

```

},
{
    "cell_type": "markdown",
    "metadata": {},
    "source": [
        "## 5. Формирование валидационной выборки\n",
        "\n",
        "Для корректной оценки качества модели важно выделить **валидационную\n",
        "выборку**,\n",
        "которая не участвует в обучении, но используется для контроля\n",
        "переобучения.\n",
        "\n",
        "Мы возьмём первые 10 000 примеров в качестве валидации."
    ]
},
{
    "cell_type": "code",
    "execution_count": null,
    "metadata": {},
    "outputs": [],
    "source": [
        "# 5. Разбиение данных на обучающую и валидационную выборки\n",
        "\n",
        "x_val = x_train[:10000]\n",
        "partial_x_train = x_train[10000:]\n",
        "\n",
        "y_val = y_train[:10000]\n",
        "partial_y_train = y_train[10000:]\n",
        "\n",
        "print(\"Обучающая выборка:\", partial_x_train.shape,\n",
        "partial_y_train.shape)\n",
        "print(\"Валидационная выборка:\", x_val.shape, y_val.shape)"
    ]
},
{
    "cell_type": "markdown",
    "metadata": {},
    "source": [
        "## 6. Построение модели нейронной сети\n",
        "\n",
        "Мы построим простую модель:\n",
        "\n",
        "- Вход: вектор длины 10 000 (one-hot представление отзыва),\n",
        "- 1-й скрытый слой: `Dense(16, activation='relu')`,\n",
        "- 2-й скрытый слой: `Dense(16, activation='relu')`,\n",
        "- Выходной слой: `Dense(1, activation='sigmoid')`.\n",
        "\n",
        "**Пояснения:**\n",
        "\n"
    ]
}

```

```

        "- `relu` (Rectified Linear Unit) – популярная нелинейная функция
активации;\n",
        "- `sigmoid` – сжимает выход в диапазон (0, 1), удобно
интерпретировать как вероятность;\n",
        "- функция потерь: `binary_crossentropy` – стандарт для бинарной
классификации;\n",
        "- оптимизатор: `rmsprop` (подходит для начала)."
```

```

    ]
},
{
    "cell_type": "code",
    "execution_count": null,
    "metadata": {},
    "outputs": [],
    "source": [
        "# 6. Определение и компиляция модели\n",
        "\n",
        "model = keras.Sequential([\n",
        "    layers.Dense(16, activation=\"relu\",
input_shape=(num_words,)),\n",
        "    layers.Dense(16, activation=\"relu\"),\n",
        "    layers.Dense(1, activation=\"sigmoid\")\n",
        "])\n",
        "\n",
        "model.compile(\n",
        "    optimizer=\"rmsprop\",\n",
        "    loss=\"binary_crossentropy\",\n",
        "    metrics=[\"accuracy\"]\n",
        ")\n",
        "\n",
        "model.summary()"
    ]
},
{
    "cell_type": "markdown",
    "metadata": {},
    "source": [
        "## 7. Обучение модели\n",
        "\n",
        "Мы обучим модель на `partial_x_train` и будем контролировать
качество на `x_val`.\n",
        "\n",
        "- Количество эпох: 20 (для иллюстрации переобучения),\n",
        "- Размер батча: 512 (компромисс между скоростью и стабильностью
градиента).\n",
        "\n",
        "Обратите внимание на изменение функции потерь и точности при
обучении."
    ]
},

```

```
{
  "cell_type": "code",
  "execution_count": null,
  "metadata": {},
  "outputs": [],
  "source": [
    "# 7. Обучение модели\n",
    "\n",
    "history = model.fit(\n",
    "    partial_x_train,\n",
    "    partial_y_train,\n",
    "    epochs=20,\n",
    "    batch_size=512,\n",
    "    validation_data=(x_val, y_val),\n",
    "    verbose=1\n",
    ")\n",
  ]
},
{
  "cell_type": "markdown",
  "metadata": {},
  "source": [
    "## 8. Визуализация процесса обучения\n",
    "\n",
    "Мы построим два графика:\n",
    "\n",
    "1. Функция потерь (loss) на обучающей и валидационной выборках.\n",
    "2. Точность (accuracy) на обучающей и валидационной выборках.\n",
    "\n",
    "По этим графикам можно оценить, начинается ли переобучение, и при  
каком числе эпох."
  ]
},
{
  "cell_type": "code",
  "execution_count": null,
  "metadata": {},
  "outputs": [],
  "source": [
    "# 8. Построение графиков потерь и точности\n",
    "\n",
    "history_dict = history.history\n",
    "\n",
    "loss = history_dict[\"loss\"]\n",
    "val_loss = history_dict[\"val_loss\"]\n",
    "acc = history_dict[\"accuracy\"]\n",
    "val_acc = history_dict[\"val_accuracy\"]\n",
    "\n",
    "epochs = range(1, len(loss) + 1)\n",
    "\n",
    "
```

```

        "plt.figure(figsize=(12, 4))\n",
        "\n",
        "# График функции потерь\n",
        "plt.subplot(1, 2, 1)\n",
        "plt.plot(epochs, loss, "o-", label="Потери на обучении")\n",
        "plt.plot(epochs, val_loss, "o-", label="Потери на\n",
валидации")\n",
        "plt.title("Функция потерь")\n",
        "plt.xlabel("Эпоха")\n",
        "plt.ylabel("Loss")\n",
        "plt.legend()\n",
        "\n",
        "# График точности\n",
        "plt.subplot(1, 2, 2)\n",
        "plt.plot(epochs, acc, "o-", label="Точность на обучении")\n",
        "plt.plot(epochs, val_acc, "o-", label="Точность на\n",
валидации")\n",
        "plt.title("Точность классификации")\n",
        "plt.xlabel("Эпоха")\n",
        "plt.ylabel("Accuracy")\n",
        "plt.legend()\n",
        "\n",
        "plt.tight_layout()\n",
        "plt.show()"
    ]
},
{
    "cell_type": "markdown",
    "metadata": {},
    "source": [
        "## 9. Анализ переобучения\n",
        "\n",
        "**Задание для студента:**\n",
        "\n",
        "1. Внимательно посмотрите на графики потерь и точности.\n",
        "2. Ответьте на вопросы:\n",
        "    - Начиная с какой эпохи функция потерь на валидации перестаёт\n",
уменьшаться и начинает расти?\n",
        "    - При каком числе эпох точность на валидации достигает\n",
максимума?\n",
        "3. Сделайте вывод: сколько эпох обучения **оптимально** использовать\n",
для данной модели?\n",
        "\n",
        "Напишите ответ в виде небольшого текста (3–5 предложений)."
    ]
},
{
    "cell_type": "markdown",
    "metadata": {},
    "source": [

```

```

        "## 10. Обучение финальной модели с оптимальным числом эпох\n",
        "\n",
        "На основе графиков выберите оптимальное число эпох (например, 4 или
5)\n",
        "и обучите **новую модель** с нуля только на `x_train`,
`y_train`,\n",
        "используя найденное количество эпох.\n",
        "\n",
        "Это позволит получить модель, которая не переобучена на валидации."
    ]
},
{
    "cell_type": "code",
    "execution_count": null,
    "metadata": {},
    "outputs": [],
    "source": [
        "# 10. Обучение финальной модели с меньшим числом эпох (пример:
5)\n",
        "\n",
        "optimal_epochs = 5 # при необходимости измените по результатам
анализа\n",
        "\n",
        "final_model = keras.Sequential([\n",
        "    layers.Dense(16, activation=\"relu\",
input_shape=(num_words,)),\n",
        "    layers.Dense(16, activation=\"relu\"),\n",
        "    layers.Dense(1, activation=\"sigmoid\")\n",
        "])\n",
        "\n",
        "final_model.compile(\n",
        "    optimizer=\"rmsprop\",\n",
        "    loss=\"binary_crossentropy\",\n",
        "    metrics=[\"accuracy\"]\n",
        ")\n",
        "\n",
        "final_model.fit(x_train, y_train, epochs=optimal_epochs,
batch_size=512, verbose=1)\n",
        "\n",
        "test_loss, test_acc = final_model.evaluate(x_test, y_test,
verbose=0)\n",
        "print(f"Точность финальной модели на тестовой выборке:
{test_acc:.4f}")
    ]
},
{
    "cell_type": "markdown",
    "metadata": {},
    "source": [
        "## 11. Прогнозирование и интерпретация результата\n",

```

```

        "\n",
        "Теперь посмотрим, какие прогнозы выдаёт модель на нескольких
        примерах из тестовой выборки.\n",
        "\n",
        "Обратите внимание: выход `sigmoid` — это число от 0 до 1, которое
        можно интерпретировать\n",
        "как **вероятность того, что отзыв положительный**."
    ]
},
{
    "cell_type": "code",
    "execution_count": null,
    "metadata": {},
    "outputs": [],
    "source": [
        "# 11. Примеры прогнозов\n",
        "\n",
        "predictions = final_model.predict(x_test[:10])\n",
        "\n",
        "for i, p in enumerate(predictions):\n",
        "    print(f"Пример {i}: предсказанное значение сигмоиды =
        {p[0]:.4f}, метка = {y_test[i]}")
    ]
},
{
    "cell_type": "markdown",
    "metadata": {},
    "source": [
        "## 12. Исследовательские задания (индивидуальная работа)\n",
        "\n",
        "Каждый студент должен выполнить два исследовательских задания
        (по выдаче преподавателя).\n",
        "Ниже приведён список вариантов. Выберите два варианта, назначенных
        Вам преподавателем,\n",
        "реализуйте изменения в отдельной копии ноутбука и кратко опишите
        результаты.\n",
        "\n",
        "**Варианты заданий:**\n",
        "\n",
        "1. Изменить размер скрытых слоёв (например, 32 и 32) и сравнить
        графики потерь и точности.\n",
        "2. Использовать активацию `tanh` вместо `relu` в скрытых слоях. Как
        это влияет на обучение?\n",
        "3. Добавить слой `Dropout(0.5)` между скрытыми слоями и сравнить
        результаты.\n",
        "4. Изменить размер словаря: `num_words = 5000` и `num_words =
        20000`. Как это влияет на качество?\n",
        "5. Изменить количество эпох обучения (например, 3, 10, 15) и
        сравнить переобучение.\n",
    ]
}

```

```

        "6. Использовать другой оптимизатор: `optimizer=\"adam\"` или
`optimizer=\"sgd\"`. Сравнить динамику обучения.\n",
        "7. Изменить размер батча (например, 128 или 1024) и оценить влияние
на процесс обучения.\n",
        "8. Изменить размер валидационной выборки (например, 5 000 или 15
000) и оценить влияние.\n",
        "9. Попробовать L2-регуляризацию в скрытых слоях
(`kernel_regularizer=keras.regularizers.l2(0.001)`).\n",
        "10. Построить confusion matrix (матрицу ошибок) для тестовой выборки
(подсказка: использовать `sklearn.metrics.confusion_matrix`).\n",
        "\n",
        "**Для каждого выполненного задания студент должен:**\n",
        "\n",
        "- кратко описать, что было изменено;\n",
        "- приложить графики (loss/accuracy);\n",
        "- дать текстовые выводы (5–10 предложений)."
```

]

```

    },
    {
        "cell_type": "markdown",
        "metadata": {},
        "source": [
            "## 13. Контрольные вопросы\n",
            "\n",
            "Ответьте письменно (в отдельной ячейке Markdown или в отчёте) на
**любые два** вопроса,\n",
            "которые задаёт преподаватель:\n",
            "\n",
            "1. Зачем необходимо векторизовать текстовые данные перед подачей в
нейронную сеть?\n",
            "2. В чём разница между функциями активации `relu` и `sigmoid`?\n",
            "3. Для чего выделяется валидационная выборка, и чем она отличается
от тестовой?\n",
            "4. Как по графику потерь (loss) можно распознать переобучение
модели?\n",
            "5. Что означает значение выхода `0.85` у модели с активацией
`sigmoid` на последнем слое?\n",
            "6. Почему для бинарной классификации обычно используют функцию
потерь `binary_crossentropy`?\n",
            "\n",
            "Ответы должны быть развёрнутыми (не в одну строку)."
```

]

```

    },
    {
        "cell_type": "markdown",
        "metadata": {},
        "source": [
            "## 14. Итог работы и оформление отчёта\n",
            "\n",
            "По итогам практического занятия студент должен подготовить:\n",

```

```

"\n",
"1. **Выполненный Jupyter Notebook**, содержащий:\n",
"  - все шаги загрузки и подготовки данных;\n",
"  - построение и обучение модели;\n",
"  - графики потерь и точности;\n",
"  - анализ переобучения;\n",
"  - реализацию двух исследовательских заданий;\n",
"  - ответы на два контрольных вопроса.\n",
"\n",
"2. **Краткий текстовый отчёт** (можно в виде отдельного файла или
итоговой ячейки Markdown), включающий:\n",
"  - цель работы;\n",
"  - краткое описание проделанных шагов;\n",
"  - основные результаты (точность на тестовой выборке);\n",
"  - выводы по исследовательским заданиям;\n",
"  - ответы на контрольные вопросы.\n",
"\n",
"На следующем занятии (4.2) мы расширим эти идеи на многоклассовую
классификацию и регрессию."
]
}
],
"metadata": {
  "language_info": {
    "name": "python",
    "pygments_lexer": "ipython3"
  },
  "orig_nbformat": 4
},
"nbformat": 4,
"nbformat_minor": 5
}

```